

Experiment 6: SELECT with BETWEEN, IN and Aggregate Functions

AIM:

To view the records from the tables using SELECT commands with BETWEEN, IN, Aggregate functions.

PROCEDURE:

- Start the program.
- Open the MySQL Command Line Client.
- Select the Supermarket database using the USE command.
- Ensure that the Vegetable table contains records.
- Use the BETWEEN clause to retrieve records within a specified range.
- Use the IN clause to retrieve records that match any value from a given list.
- Use Aggregate Functions (COUNT, SUM, AVG, MAX, MIN) to perform calculations on table data.
- Display the results.
- Stop the program.

SQL Queries

1. BETWEEN Clause

Display vegetables with price between 30 and 40

```
SELECT * FROM Vegetable  
WHERE Price BETWEEN 30 AND 40;
```

```
mysql> SELECT * FROM Vegetable  
-> WHERE Price BETWEEN 30 AND 40;  
+-----+-----+-----+-----+-----+  
| VegID | VegName | Category | Price | Stock |  
+-----+-----+-----+-----+-----+  
| 101 | Tomato | Vegetable | 30.50 | 100 |  
| 103 | Carrot | Root | 40.00 | 80 |  
| 104 | Onion | Vegetable | 35.00 | 150 |  
+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Display vegetables with stock between 80 and 150

```
SELECT * FROM Vegetable  
WHERE Stock BETWEEN 80 AND 150;
```

```
mysql> SELECT * FROM Vegetable  
-> WHERE Stock BETWEEN 80 AND 150;  
+-----+-----+-----+-----+-----+  
| VegID | VegName | Category | Price | Stock |  
+-----+-----+-----+-----+-----+  
| 101 | Tomato | Vegetable | 30.50 | 100 |  
| 103 | Carrot | Root | 40.00 | 80 |  
| 104 | Onion | Vegetable | 35.00 | 150 |  
| 105 | Brinjal | Vegetable | 45.50 | 90 |  
+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

2. IN Clause

Display vegetables whose category is Vegetable or Root

```
SELECT * FROM Vegetable  
WHERE Category IN ('Vegetable','Root');
```

```
mysql> SELECT * FROM Vegetable  
-> WHERE Category IN ('Vegetable','Root');  
+-----+-----+-----+-----+-----+  
| VegID | VegName | Category | Price | Stock |  
+-----+-----+-----+-----+-----+  
| 101 | Tomato | Vegetable | 30.50 | 100 |  
| 102 | Potato | Vegetable | 25.00 | 200 |  
| 103 | Carrot | Root | 40.00 | 80 |  
| 104 | Onion | Vegetable | 35.00 | 150 |  
| 105 | Brinjal | Vegetable | 45.50 | 90 |  
+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```

Display selected vegetables by name

```
SELECT * FROM Vegetable
WHERE VegName IN ('Tomato','Onion','Carrot');
```

```
mysql> SELECT * FROM Vegetable
-> WHERE VegName IN ('Tomato','Onion','Carrot');
+-----+-----+-----+-----+-----+
| VegID | VegName | Category | Price | Stock |
+-----+-----+-----+-----+-----+
| 101   | Tomato  | Vegetable | 30.50 | 100   |
| 103   | Carrot  | Root      | 40.00 | 80    |
| 104   | Onion   | Vegetable | 35.00 | 150   |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Aggregate Functions

COUNT()

Count total number of vegetables

```
SELECT COUNT(*) AS Total_Vegetables
```

```
FROM Vegetable;
```

```
mysql> SELECT COUNT(*) AS Total_Vegetables
-> FROM Vegetable;
+-----+
| Total_Vegetables |
+-----+
| 5                 |
+-----+
1 row in set (0.01 sec)
```

SUM()

Calculate total stock

```
SELECT SUM(Stock) AS Total_Stock
```

```
FROM Vegetable;
```

```
mysql> SELECT SUM(Stock) AS Total_Stock
-> FROM Vegetable;
+-----+
| Total_Stock |
+-----+
|          620 |
+-----+
1 row in set (0.00 sec)
```

AVG()

Calculate average price

```
SELECT AVG(Price) AS Average_Price
```

```
FROM Vegetable;
```

```
mysql> SELECT AVG(Price) AS Average_Price
-> FROM Vegetable;
+-----+
| Average_Price |
+-----+
|    35.200000 |
+-----+
1 row in set (0.00 sec)
```

MAX()

Find the highest price

SELECT MAX(Price) AS Highest_Price

FROM Vegetable;

```
mysql> SELECT MAX(Price) AS Highest_Price
-> FROM Vegetable;
+-----+
| Highest_Price |
+-----+
|          45.50 |
+-----+
1 row in set (0.03 sec)
```

MIN()

Find the lowest price

SELECT MIN(Price) AS Lowest_Price

FROM Vegetable;

```
mysql> SELECT MIN(Price) AS Lowest_Price
-> FROM Vegetable;
+-----+
| Lowest_Price |
+-----+
|          25.00 |
+-----+
1 row in set (0.00 sec)
```

Combined Aggregate Query

Display all aggregate values together

SELECT

COUNT(*) AS Total_Items,

SUM(Stock) AS Total_Stock,

AVG(Price) AS Average_Price,

MAX(Price) AS Highest_Price,

MIN(Price) AS Lowest_Price

FROM Vegetable;

```
> FROM Vegetable,
+-----+-----+-----+-----+-----+
| Total_Items | Total_Stock | Average_Price | Highest_Price | Lowest_Price |
+-----+-----+-----+-----+-----+
|          5 |         620 |    35.200000 |         45.50 |         25.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```